

Dump and Analysis of Android Volatile Memory on Wechat

Fan Zhou*, Yitao Yang*, Zhaokun Ding[†], Guozi Sun*

*College of Computer, Nanjing University of Posts and Telecommunications, Nanjing 210003, China

[†]Institute of Command Information System, PLA University of Science and Technology, Nanjing 210007, China
yutruth123@gmail.com, youngyt@gmail.com, dzkad@netease.com, sun@njupt.edu.cn

Abstract—With the popularity of smartphones, various types of mobile crimes emerge endlessly. Evidence from mobile phones is mostly obtained by non-volatile physical memory dump and file system analysis. The two methods can extract lots of private data, but often invalid for encrypted and deleted data. In this paper, we discuss the Android volatile memory and introduce some methods to dump the memory. Analysis on the Android volatile memory are also presented using software tools. At last the paper provides an in-depth analysis of Android memory structures to extract the encrypted chats and deleted messages on a popular social network application called Wechat [1]. The results show that all chats can be extracted in the form of plaintext, including some deleted messages.

Keywords—Android forensics; Volatile memory acquisition; Memory analysis; RAM; Wechat

Corresponding author: Guozi Sun, sun@njupt.edu.cn.

I. INTRODUCTION

Nowadays much private information is stored on smart phones, including photos, messages, chats, web browse histories, GPS data etc. This has brought huge demand of digital forensics. For example, acquiring the user's chat histories, entering a locked screen terminal and retrieving the users deleted messages are some interesting topics. While a few research reports have discussed the analysis of Android file system and non-volatile memory [2], little on the acquisition and analysis of Android volatile memory. Android file system research is mainly on extracting the information stored in the sqlite [3] database, but often invalid for encrypted information unless the encryption key is known. The Android physical image forensics can retrieve some of the deleted messages, but now with the rapid development of storage performance, the storage size is significantly improved, so dumping the complete physical image may cost lots of time [4].

Android memory preserves structured and unstructured information, like the application loaded data, processes opened files, connected network activities etc. For encrypted data and temporary files, memory analysis is the only method to obtain the evidence. Although the analysis of the computer's memory has been investigated for several years [5] [6], the analysis of Android volatile memory is still at the beginning.

This paper introduces some methods to acquire and analyze volatile memory from Android devices. We compare several current methods of acquiring Android memory, and use lime

[7] to dump memory. Through the memory analysis, submitted credentials are checked if they could be recovered from the memory by Winhex [8] tool. Then userland activities are listed with Volatility, the Volatility Memory Analysis framework [9]. Finally, a python script is presented to extract the encrypted chats and deleted messages on a popular social networking application called Wechat.

In the rest of this paper, Section II introduces Android background and experimental environment. Section III describes related work. Section IV introduces the acquisition of Android memory and Section V presents analysis of Android memory. Section VI summarizes our conclusions and future work.

II. BACKGROUND

A. Android Mobile Operating System

Android is an open source Operating System (OS) developed by Google for mobile devices. It is based on Linux 2.6 kernel. All higher layers rely on the kernel's core services such as security, memory management, process management etc. To run applications, Android uses its own Virtual Machine called Dalvik [10]. Each Android application runs in a separate process within its own Dalvik instance. However, as the latest Android 5.0 system comes out, ART has gradually replaces the Dalvik and becomes the default application startup mode.

B. Different Types of Memory

Android devices are equipped with different types of memory for different functions. One is volatile physical memory (RAM) which will lose its data when power is switched off. The general capacity of RAM is often 1G or 2G. Some researchers think the data lost related to the temperature. According to [11], if the temperature below 10, then the data will be lost at a very slow speed. To exploit this behavior in the digital investigation [12] [13], the recovery tool FROST (Forensic Recovery of Scrambled Telephones) [14] has been developed. Another type of memory is based on the internal storage of NAND flash memory technology called non-volatile memory [15], it does not require power to save data and its capacity is very large. But the data stored in this memory is often encrypted. Android also has built-in or external SD card to store application data and media files.

In many cases the private information cant be acquired from the non-volatile memory, such as the user's login account and password. Some information especially the data of finance,

shopping and social network applications is often not stored in the non-volatile memory. These applications submit certificates, which only spread in the volatile memory (RAM). In [16] [17], the authors have demonstrated that lots of private information including user's login account and password can be found in the volatile memory. The data stored in the non-volatile memory may be encrypted, for example, social network applications such as QQ, Wechat, encrypt all the chats in the database, and Android 4.0 system also support disk encryption completely [18]. But in RAM, these data is stored in plaintext, one reason is that encryption can affect memory access rate and bring user very inconvenient experience. For the forensic researchers, acquiring evidence from Android RAM is a good choice, especially considering that users now rarely shut off the devices so the applications are running in the background all the time.

C. Test Setup and Development Environment

To perform the investigations and tasks outlined in this paper, we make use of a Samsung Galaxy S2 (I9100G) smartphone that runs Android version 4.0.3, also known as Ice Cream Sandwich. All findings and used tools will be based on this version. In case of Volatility, a branch called 2.3-devel will be used. This branch already includes the ARM support targeted for the final release 2.3. The kernel module lime [7] used in this paper is based on the version 1.1. The paper introduce the acquisition of the Android volatile memory and analyze memory using Winhex, Volatility tools etc. Finally a python script is presented to extract the user's chat records and retrieve some deleted messages of Wechat.

III. RELATED WORK

Traditional access to memory can be based on hardware and software. JTAG and FireWire are the mainly hardware methods and the software methods mainly depend on fmem and dd tools [19]. Though the analysis on memory structures has been researched for a few years, they are mostly on x86 architecture of Windows [5] and Linux operating system [20], little on Android. Sylve et al. mainly introduced how to dump the memory from Android system [21] and compared several methods on the performance of the memory integrity. The results of their work are shown in TABLE I. Fmem and dd tools can acquire memory successfully on the Linux operating system, but their performance on Android is poor. This is mainly due to some of the Android kernel security mechanism that prevent the kernel to load external code. Fmem and dd creates a character device to read the Android physical memory which is considered unsuitable, because it makes use of kernel functions that are not available on the ARM platform.

Lime [7] is an open source tool for dumping memory from Android devices developed by Sylve et al. It can be loaded into Linux operating system kernel, such as various Android devices. Lime support two ways to acquire Android memory, one is via TCP network, and the other is through the local storage as the SD card. In TABLE I, the results show that the Android memory integrity can reach above 99% by lime.

TABLE I: Memory Dump Integrity Comparison

Method	Total number of pages	Number of identical pages	Percentage of identical pages
lime(TCP)	131,072	130,365	99.46%
lime(SD Card)	131,072	129,953	99.15%
fmem(SD Card)	131,072	105,080	80.17%

DDMS [22] is also a method to dump Android memory, but it cannot acquire the complete physical memory, it can only acquire a single process heap contents. Leppert et al. [23] used DDMS and MAT [24] tools to analyze Android processes heap data, but the experiment was based on the Android version 2.3 and tested only on the Android emulator.

DDMS and MAT can be used to capture and analyze process memory, but it is quite inadequate in this way. Firstly DDMS is only available in debug mode, developers often close the mode after the mobile phone leave the factory. Secondly, DDMS can only dump a single process heap memory, rather than a complete physical memory so that it loses lots of data. In this paper we dump a complete Android physical memory by lime.

Some third-party organizations developed Volatility [9] plugins for Android Dalvik and application analysis. In [11] [25] [26], the authors mainly discussed the analysis of the Android Dalvik and applications. Although the research above can acquire the application data from the memory dump, their methods normally only for open source applications. For those applications such as Wechat, QQ, we are unable to get the data structures. Meanwhile, with the development of Android 5.0, Dalvik is not the default startup mode for application, so the work to analyze Android memory from Dalvik may not be feasible.

IV. ACQUIRE ANDROID PHYSICAL MEMORY

A. Limitations

Because Android neither provide hardware equipment nor APIs to dump memory. Loading the code in the kernel mode to dump a complete copy of memory is the only method. While this approach is not idealized, it is very common. Loading codes in the kernel module need to break through some Android security mechanism, so root permission should be got first.

Rooting the Android mobile phone to meet the personalized demand is very common today. Now lots of tools can root a mobile phone. In addition to get root access, the bootloader should also be unlocked to install our custom kernel. The kernel code of different mobile manufacturers is different, so we're going to analyze for a certain mobile phone.

B. Dump Physical Memory with Lime

Sylve et al. develop lime in 2011 and lime is the only method can completely acquire memory from Android so far. In order to acquire a complete physical memory, lime uses the Android iomem structure in the kernel, each iomem structure with a list of the start and end element marked the beginning and end of the physical memory address.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
06EF8D00	FC	4C	FD	F8	00	00	00	00	00	00	00	00	01	3D	30	26	鞠=0&
06EF8D10	10	DB	D4	01	48	46	0D	40	B5	6E	42	E5	00	00	00	00	.?? HF.0祔B?....
06EF8D20	00	00	00	00	01	33	30	30	10	DB	D4	01	48	46	0D	40	300.?? HF.0
06EF8D30	87	F0	D8	E8	00	00	00	00	00	00	00	00	01	00	00	00	国国.....
06EF8D40	00	F4	D3	01	00	00	00	00	00	00	00	00	00	00	00	00	.??
06EF8D50	00	00	00	00	01	2F	61	70	38	70	D5	01	85	A4	8D	40	/ap8p? 参.0
06EF8D60	00	00	00	00	00	00	00	00	00	00	00	00	01	78	3F	73	x?s
06EF8D70	63	68	6F	6F	6C	69	64	3D	31	34	34	39	26	75	73	65	choolid=1449&use
06EF8D80	72	54	79	70	65	3D	30	26	75	73	65	72	6E	61	6D	65	rType=0&username
06EF8D90	3D	31	32	31	31	30	31	33	30	34	31	30	31	33	30	30	0000000000000000
06EF8DA0	26	70	61	73	73	77	6F	72	64	3D	0000000000000000						&password= 00000000
06EF8DB0	00	00	00	00	00	00	00	00	10	2E	CA	01	00	00	00	00	?
06EF8DC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Fig. 1: Search password in memory by Winhex.

To dump memory by lime, we have to make some preparations. This paper experiments on Samsung I9100G mobile phone whose kernel source code can be downloaded on Samsung official website. So firstly download the source code on Samsung official website. Then compile the kernel source code as Listing 1 showing:

Listing 1. Compile the kernel source code

```
$cd $KSRC_PATH
$make ARCH=arm
CROSS_COMPILE=$CC_PATH/arm-eabi-modules_prepare
```

Modify lime configuration file and compile lime as Listing 2 showing below:

Listing 2. Modify lime configuration file and compile

```
obj-m:= lime.o
lime-objs:= main.o tcp.o disk.o
KDIR:= /path/to/kernel-source
PWD:= $(shell pwd)
CCPATH:=/path/to/Android-ndk/toolchains/arm-linux-
Androideabi-4.4.3/prebuilt/linux-x86/bin/default:
$(MAKE) ARCH=arm CROSS_COMPILE=$(CCPATH)/arm-
eabi- -C $(KDIR) M=$(PWD) modules
$ cd $LIME_SRC
$ make
```

Finally, enter into the terminal and type the commands showed in Listing 3 to acquire Android physical memory.

Listing 3. Acquire physical memory by lime

```
$ adb push lime.ko /sdcard/lime.ko
$ adb shell
$ su
#insmod/sdcard/lime.ko path=/sdcard/ram.lime format=lime
```

Once the acquisition process is completed, we can power down the phone, remove the SD card, and transfer the memory dump to the examination machine. If the phone cannot be

powered down, adb can also be used to transfer the memory dump to the investigator's machine. The available memory acquisition method for mobile devices is not perfect now and continues to be an area for further research.

V. ANALYZE ANDROID MEMORY

A. Find Strings with Winhex

After acquiring Android physical memory successfully, then the easiest analysis method is to find strings in the memory such as the username, password etc. The paper tested some applications belong to different categories. In the experiments we performed the following steps: First, install the applications under investigation for the various versions of the Android OS. Next, open an application randomly and submit the credentials (i.e.username and/or password) to log in. Then set the application process to run in the background. This was done because most users close the applications directly instead of logging out. After this, dump the volatile memory and search the strings in the memory dump. Although the results in [16] indicate that most applications store the users account and password in the memory, our results show that a mature application generally does not directly transmit passwords in plain text and the password is encrypted in the memory. But there are some applications such as nupter and library management application that directly transmit the account and password in memory, as shown in Figure 1 we can find the username and password successfully by Winhex.

Although Winhex can find strings successfully, this method is very limited as it can be used for any disk forensics and does not focus on OS or application data structures. And if we obtain a mobile phones memory dump, the strings to be searched is often unknown.

B. Use Volatility Plugins

Volatility, a volatile memory artifact extraction utility framework that is completely open source, released under the

```
yutruth@ubuntu:~/volatility$ python vol.py -f ~/0410.lime --profile=LinuxI9100G-1013ARM linux_pslist
```

Volatility Offset	Foundation Name	Volatility Pid	Framework Uid	2.3 Gid	DTB	Start Time
0xec444000	init	1	0	0	0xaba6c000	2014-04-10 04:03:36 UTC+0000
0xec444440	kthreadd	2	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec444880	ksoftirqd/0	3	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec445540	migration/0	6	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec445980	watchdog/0	7	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec447300	khelper	12	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec447740	netns	13	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec447b80	suspend_sys_syn	14	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec494000	suspend	15	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec494440	sync_supers	16	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec494880	bdi-default	17	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec494cc0	kblockd	18	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec495100	omap2_mcspi	19	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec495540	khubd	20	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec495980	twl6030-irq	21	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec495dc0	irq/374-TWL6030	22	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec496640	cfg80211	24	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec496a80	musb-otg	25	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec496ec0	kinteractiveup	26	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec497300	khungtaskd	27	0	0	-----	2014-04-10 04:03:36 UTC+0000
0xec497740	kswapd0	28	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xec497b80	fsnotify_mark	29	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebef0000	ecryptfs-kthrea	30	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebef0440	crypto	31	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebef3b80	hdmi_hotplug	44	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebf34000	hdcp	45	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebf34440	dsscomp_apply	46	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebf34880	dsscomp_apply	47	0	0	-----	2014-04-10 04:03:37 UTC+0000
0xebf34cc0	dsscomp_apply	48	0	0	-----	2014-04-10 04:03:37 UTC+0000

Fig. 2: List all the running processes using linux pslist.

GNU General Public License and written in Python. Given a memory image, Volatility can extract running processes, opened network sockets, memory maps for each process, and kernel modules.

Before a memory can be analyzed using Volatility, a Volatility profile that is passed to the Volatility framework as a command line parameter must be created. Basically, a profile is a compressed archive containing two files: System.map and module.dwarf. The System.map file contains the symbol names and addresses of static data structures in the Linux kernel. The module.dwarf file emerges by compiling a module against the target kernel and extracting the DWARF debugging information from it. DWARF [27] is a standardized debugging format used by source level debuggers to establish a logical connection between an output binary and its actual source code. Creation of the actual profile by running one command: `zip profile.zip mod_path.dwarf sys_path.map`. The resulting ZIP-file needs to be copied to the Volatility source tree, and then the profile shows up in the profiles section of the Volatility help output. We are now able to use Volatility together with Android memory dumps.

Volatility has developed many kernel-level plugins for analysis of memory, but for Android, we are more concerned with the application data, such as chat history, deleted message that is not provided by Volatility. As shown in Figure 2, a Volatility plugin is used to list all the current running processes in the mobile phone. Although Android support in Volatility is quite new, Linux support is not. So a number of corresponding Linux plugins are already available. The `linux_pslist` plugin enumerates all the running processes in the system, similar to the Unix command `ps`.

C. Extract Wechat Records with Python Script

Faced with Winhex and Volatility limitations, this paper proposed a new method using python scripts directly to do

memory analysis. We mainly researched on Wechat, a popular social network application that has the largest number of users in Asia and the registered users have reached 600 million on November 2013. Our main task is to extract all user's sent and received chat histories and retrieve deleted messages as much as possible.

Figure 3 shows the chat record of Wechat searched in the memory. The first six bytes "01454A421E77" is a timestamp. "yutruth" is sender ID. "this is test5" is the chat content and "{" is the ending tag. According to these characteristics, this paper write a script to analyze the whole memory to extract all the chat histories. To separate sender ID and chat content, we use a db file called IndexMicroMsg under the path `/data/data/com.tencent.mm/MicroMsg`. The `aux_index` column in IndexMicroMsg.db stores a users friends ID as Figure 4 shows.

The paper do the following experiments before writing a python script to extract chat records:

- Send and receive twenty chat records of Wechat (including text, image, location and voice formats), leaving the application running in background and then dump physical memory by lime.
- Send and receive twenty chat records of Wechat (including text, image, location and voice formats), log out the application and then dump physical memory by lime.
- Send and receive twenty chat records of Wechat (including text, image, location and voice formats), restart the phone and log in Wechat and then dump physical memory by lime.
- Delete twenty chat records of Wechat that only including text format, leave the application running in background and then dump physical memory by lime.
- Delete twenty chat records of Wechat that only including text format, restart the phone and log in Wechat, then

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
0241DE80	01	05	1B	2D	00	00	48	3C	EF	AC	F0	01	03	00	00	01	...-..H<铿?
0241DE90	45	4B	B9	9D	F8	79	75	74	72	75	74	68	41	6E	64	72	EK? 驚uttruthAndr
0241DEA0	6F	69	64	20	66	6F	72	65	6E	73	69	63	7B	00	00	00	oid forensic{...
0241DEB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0241DEC0	00	00	00	00	01	00	00	00	00	7D	4D	18	0D	00	04	01	}M.....
0241DED0	01	01	01	05	1B	27	00	00	48	25	18	28	ED	01	02	01	'..H%. (? ..
0241DEE0	00	01	45	4A	42	1E	77	79	75	74	72	75	74	68	74	68	..EJB.wyuttruthth
0241DEF0	69	73	20	69	73	20	74	65	73	74	35	7B	00	00	00	00	is is test5{....

Fig. 3: The chat record of Wechat searched in physical memory.

dump physical memory by lime.

#	d..	type	subtype	entity_id	aux_index	timestamp	statu
1	1	65536	0	5	weixin	1396797455000	0
2	2	131074	1	31	gh_22b87fa7cb3c	1396797486324	0
3	3	131074	3	31	gh_22b87fa7cb3c	1396797486324	0
4	4	131074	9	31	gh_22b87fa7cb3c	1396797486324	0
5	5	131074	10	31	gh_22b87fa7cb3c	1396797486324	0
6	6	131072	1	32	assassfly	1396797486335	0
7	7	131072	3	32	assassfly	1396797486335	0
8	8	131072	9	32	assassfly	1396797486335	0
9	9	131072	10	32	assassfly	1396797486335	0
10	10	131072	1	33	nuoery	1396797486342	0
11	11	131072	3	33	nuoery	1396797486342	0
12	12	131072	9	33	nuoery	1396797486342	0
13	13	131072	10	33	nuoery	1396797486342	0
14	14	131072	2	33	nuoery	1396797486342	0

Fig. 4: The contents of IndexMicroMsg.db.

After analyze the five categories of experiments by Winhex, the results are shown in TABLE II. The paper does experiments of chat records in plain text, image, location and the voice formats. The results show that as long as the user does not completely log out of the application, all these chat records are stored in memory, for the deleted chats, although not every chat can be retrieved, most of the chats can be found in memory.

TABLE II: Memory Dump Integrity Comparison

Serial Number	Text Found	Image Found	Location Found	Voice Found
Scenario1	10/10	3/3	4/4	3/3
Scenario2	0/10	0/3	0/4	0/3
Scenario3	10/10	3/3	4/4	3/3
Scenario4	15/20	None	None	None
Scenario5	15/20	None	None	None

Finally, we use python to write an automatic script to extract all the chat records, the script can be used quickly in memory searching and output all the chat records including sent or received time, the sender or receiver, chat content etc. The start and end time are set according to the users input. The

```
C:\Users\Administrator\workspace\forensic\wechat>python wechat_forensic.py
```

```
begin time:2014-07-05 00:00:00
```

```
end time:2014-07-07 00:00:00
```

Chat Time	Sender and Receiver	Chat Content
2014-07-05 11:09:56	3399456012@chatroom yuttruth	中午
2014-07-05 11:10:11	3399456012@chatroom yuttruth	我也醒
2014-07-05 11:10:40	3399456012@chatroom yuttruth	在睡
2014-07-05 11:11:31	3399456012@chatroom yuttruth	xia wu 2 dian zou
2014-07-05 11:13:41	3399456012@chatroom yuttruth	sui bian mai dian chi
2014-07-05 11:14:58	3399456012@chatroom yuttruth	hahaha
2014-07-05 11:15:10	3399456012@chatroom yuttruth	周帆又在做实验
2014-07-05 11:15:14	3399456012@chatroom yuttruth	哈哈
2014-07-05 11:18:47	3399456012@chatroom yuttruth	好
2014-07-05 11:19:04	wxid_1555065551922 yuttruth	chi fan qu le?
2014-07-05 11:19:43	wxid_1555065551922 yuttruth	
2014-07-05 11:19:49	wxid_1555065551922 yuttruth	看不懂
2014-07-05 11:23:39	3399456012@chatroom yuttruth	你要做神马呢
2014-07-05 11:24:52	3399456012@chatroom yuttruth	和男票吃
2014-07-05 11:26:05	3399456012@chatroom yuttruth	吃完饭就没事

Fig. 5: The script output the chat records in Chinese.

```
begin time:2014-07-07 00:00:00
```

```
end time:2014-07-09 00:00:00
```

Chat Time	Sender and Receiver	Chat Content
2014-07-07 06:43:12	yuttruth ztatf123	Hi, how are you recently?
2014-07-07 06:47:48	yuttruth ztatf123	Yeah, that is good !
2014-07-07 06:52:51	yuttruth ztatf123	I am busy recently. I find i have a lot to learn
2014-07-07 06:53:07	yuttruth ztatf123	The weather is nice today
2014-07-07 07:02:34	yuttruth ztatf123	So when will you come back?
2014-07-07 07:04:23	yuttruth ztatf123	OK.see you then
2014-07-07 07:07:42	yuttruth ztatf123	回了
2014-07-07 07:08:23	yuttruth ztatf123	好的
2014-07-07 07:08:31	yuttruth ztatf123	你最近忙啥
2014-07-07 07:09:42	yuttruth ztatf123	嗯哪, 我也是。
2014-07-07 07:10:20	yuttruth ztatf123	最近在研究算法
2014-07-07 07:14:15	yuttruth ztatf123	对,什么时候回来一起吃顿饭
2014-07-07 07:14:50	yuttruth ztatf123	好的, 到时候联系
2014-07-07 07:15:01	yuttruth ztatf123	先这样了, 回聊

Fig. 6: The script output the chat records in English and Chinese.

chats are using Chinese and English language. Figure 5 and Figure 6 are part of the experimental results, from the output

we can also find most of the deleted messages.

VI. CONCLUSION AND FUTURE WORK

With the development of smartphones, the traditional physical image and file system forensics cannot meet the requirements of forensic analyst. The memory forensics is becoming more and more important. This paper mainly introduced some methods and focused on lime to acquire Android complete physical memory. Then we analyzed Android memory using Winhex and Volatility tools. Some credentials can be found from the memory by Winhex, and Volatility can extract running processes, opened network sockets, memory maps for each process etc. But the strings to be searched by Winhex is often unknown and creating a Volatility profile for Android is difficult, so we presented an in-depth analysis of Android memory structures. We did some experiments and tested on Wechat, a popular social networking application. Our task was to extract the encrypted and deleted messages. Finally the results show that all chats can be extracted in plaintext, including some deleted messages.

Although physical memory analysis is very important for the forensic analyst, we should also see the limitation of the Android physical memory forensics. A problem that we have to address is the fact that the majority of mobile devices are shipped with a custom version of the Android OS designed from the mobile device manufacturers. So the memory dumps can be different for each Android device. Also obtaining the Android root access have to own mobile phones and open USB debugging function etc. With the development of Android 5.0, Dalvik is not the default startup mode for the application, so the work to analyze Android memory from Dalvik may not be feasible.

As a future work, we want to develop a generic kernel module that can be loaded into any Android kernel without additional cross-compilation steps. In this paper, our main work is to extract chat records from the memory. Next we are going to extract the encrypted key of application and expand the analysis to other applications, such as QQ, whatsapp, line. As ART mode gradually replaces Dalvik, how to analyze the application data with ART startup mode is also our work.

ACKNOWLEDGEMENT

The authors would like to thank the reviewers for their detailed reviews and constructive comments. This paper is supported by the Chinese National Natural Science Foundation (No. 61373006), the Foundation of Nanjing University of Posts and Telecommunications (No. NY213160).

REFERENCES

- [1] <http://www.wechat.com/en/>.
- [2] A.Hoog, "Android Forensics: Investigation, Analysis and Mobile Security for Google Android," Syngress Publishing, 1st edition, 2011.
- [3] <http://www.sqlite.org/>.
- [4] A.Walters and N.Petroni, Volatools: Integrating volatile memory forensics into the digital investigation process. Digital Investigation, pages 1C18, 2007.
- [5] T.Vidas, "Post-Mortem RAM Forensics (or Reversing Windows RAM after-the-fact)," in CanSecWest07. Vancouver, Canada: Nebraska University, Apr. 2007.
- [6] A.Schuster, The impact of microsoft windows pool allocation strategies on memory forensics. Digital Investigation, Supplement(0):S58 C S64, 2008. The Proceedings of the Eighth Annual DFRWS Conference.
- [7] J.Sylve, "LiME - Linux Memory Extractor," in ShmooCon12. Washington, D.C.: Digital Forensics Solutions, LLC, Jan. 2012.
- [8] <http://www.winhex.com/index-c.html>.
- [9] Volatilitysystems.[Online].Available:<https://code.google.com/p/Volatility>.
- [10] <https://source.android.com/devices/tech/dalvik/>.
- [11] C.Hilgers and H.Macht, "Post-Mortem Memory Analysis of Cold-Booted Android Devices", Jan. 2014.
- [12] R.Carbone, C.Bean and M.Salois, "An in-depth analysis of the cold boot attack," DRDC Valcartier, Defence Research and Development, Canada, Tech. Rep., Jan. 2011, Technical Memorandum.
- [13] J.A.Halderman, S.D.Schoen, N.Heninger, W.Clarkson, W.Paul, J.A.Calandrino, A.J.Feldman, J.Appelbaum and E.W.Felten, "Lest We Remember: Cold Boot Attacks on Encryptions Keys," in Proceedings of the 17th USENIX Security Symposium, Princeton University. San Jose, CA: USENIX Association, Aug. 2008, pp. 45C60.
- [14] T.Muller and M.Spreitzenbarth, "FROST: Forensic Recovery of Scrambled Telephones," in International Conference on Applied Cryptography and Network Security (ACNS). Banff, Alberta, Canada: Rei Safavi-Naini and Michael Locasto, Jun. 2013. Investigation, vol. 8, no. 3-4, pp. 175 C 184, 2012.
- [15] S.Y.Willassen, "Forensic analysis of mobile phone internal memory," In Mark Pollitt and Sujeet Sheno, editors, IFIP Int. Conf. Digital Forensics, pages 191C204. Springer, 2005.
- [16] D.Apostolopoulos, G.Marinakis and C.Xenakis, "Discovering authentication credentials in volatile memory of Android mobile devices" 2013.
- [17] S.Karayianni, V.Katos and C.K.Georgiadis, "A framework for password harvesting from volatile memory", International Journal of Electronic Security and Digital Forensics, Vol. 4, No. 2-3, pp.154C163, 2012.
- [18] A.Skillen, D.Barrera, and P.V.Oorschot, "Deadbolt: Locking Down Android Disk Encryption," in 3rd Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices (SPSM 2013), Berlin, Germany, Nov. 2013.
- [19] J.Sylve, "Android Mind Reading: Memory Acquisition and Analysis with DMD and Volatility", Digital Forensics Solutions, LLC, Jan. 2012.
- [20] J.M.Urrea, "An Analysis of Linux RAM Forensics," Masters thesis, Naval Postgraduate School, Monterey California, 2006.
- [21] J.Sylve, A.Case, L.Marziale and G.G.Richard, "Acquisition and analysis of volatile memory from android devices," Digital Investigation, vol. 8, no. 3-4, pp. 175 C 184, 2012.
- [22] <http://developer.android.com/tools/debugging/ddms.html>.
- [23] S.Leppert, "Android Memory Dump Analysis," Student Research Paper, Chair of Computer Science 1 (IT-Security), Friedrich-Alexander-University Erlangen-Nuremberg, Germany, 2012.
- [24] <http://www.eclipse.org/mat/>.
- [25] H.Macht, "Live Memory Forensics on Android with Volatility", Jan. 2013.
- [26] A.Case, "Memory Analysis of the Dalvik (Android) Virtual Machine," in SOURCE Seattle 2011, presentation video and slides, 2011.
- [27] M.J.Eager and E.Consulting, "Introduction to the DWARF Debugging Format," Group, 2007.